PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
August to December 2026

UE24CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

PantryChef
Smart Recipe Finder from Everyday Ingredients

Submitted by:

PES1UG24AM304	Tarun Jagan	Semester 5
PES1UG24AM301	Tanishk Gowda	Semester 5

Class of Prof. ***Mir Junaid Rasool***
Lab Faculty Prof Mir Junaid Rasool

Abstract

PantryChef is a recipe discovery application that helps users decide what to cook from ingredients already available at home. It combines a Next.js web interface and API with PostgreSQL relational storage and pgvector semantic retrieval. Users enter pantry ingredients, browse dietary and cuisine filters, search by keywords or meaning, inspect missing ingredients, and save or rate recipes. Pantry searches return only recipes for which at least 75% of required ingredient names are available; the ready-to-cook filter requires a complete match.

The current public catalogue contains 545 approved, non-synthetic recipes, including 500 attributed Wikibooks imports and 45 established dishes. The supported meat and seafood categories are chicken, fish, mutton, beef, lamb, prawns and lobster. Imported ingredient measurements, cooking steps and source attribution are retained. Each public recipe has a distinct locally hosted food photo, labeled as a serving suggestion because the photographed presentation may differ from the exact recipe.

The relational model uses normalized recipe, ingredient and user records with associative relations for pantry entries, recipe ingredients and saved recipes. Primary and foreign keys, uniqueness and check constraints protect data integrity. Stored routines compute pantry coverage, a trigger maintains rating averages, and transactions keep related edits atomic. Full-text and ingredient indexes support structured search. A fitted TF-IDF and TruncatedSVD model produces 128-dimensional vectors for cosine similarity search.

Authentication distinguishes Regular User and Admin access. Session-derived database policies restrict records by ownership and role. The latest automated validation records 18 passing tests covering API workflows, database authorization, vector changes, source catalogue integrity, photo coverage and pantry search boundaries. This report presents the design, commands, results and application evidence for the DBMS Level 3 project.

Table of Contents

Sl No	Topic	Page No
1	Introduction	4
2	User Requirements Specification and Technology Stack	5
3	ER Model	6
4	Relational Model	8
5	SQL DDL Statements	11
6	SQL DML Statements	13
7	NoSQL Vector Database Commands	16
8	Results	18
9	Next.js Project Tree Structure	20
10	Application Code and Demo	21
11	Evaluation Rubrics Completion Status	24

Introduction

Problem Statement

People often have several ingredients at home but lack a practical way to identify suitable meals. Browsing a conventional recipe collection can return dishes that require many additional ingredients. Free-text cravings also differ from stored recipe titles, and loosely interpreted meat names can produce unrelated results. PantryChef addresses these problems through required-ingredient matching, ingredient-aware search and semantic retrieval.

Objectives and scope

The application ranks recipes by available required ingredients, shows missing items, supports cuisine and dietary filters, and preserves personal pantries and cookbooks. Keyword search uses full-text indexing; Meaning search uses vector distance with explicit ingredient guards. A search for fish therefore requires a real required fish ingredient rather than a coincidental word such as fish sauce.

The public collection is separate from the 900 synthetic benchmark records retained in the database. Historical performance evidence was captured with 100,000 synthetic recipe rows and 699,991 ingredient links; those measurements demonstrate index behavior and are not a claim that the public application contains 100,000 distinct real recipes.

System architecture

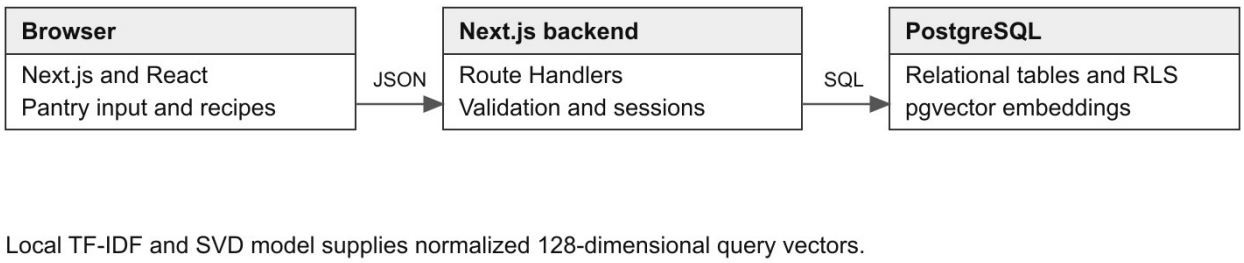


Figure 1 Browser requests pass through the Next.js API to relational and vector data.

PostgreSQL stores related data and vectors in the same transaction boundary. Python trains the local embedding model during setup; the running Node.js backend uses the exported model to embed queries. Credentials remain in the ignored local environment file, while the browser receives only application JSON.

User Requirements Specification and Technology Stack

Role	Supported actions
Visitor	Browse approved recipes; select ingredients; search and filter; read methods and photo/source credits.
Regular User	Register and sign in; save pantry; save recipes; add, edit and remove own ratings; submit a recipe for moderation.
Admin	Review submissions; approve or reject; add, edit and delete recipes; inspect aggregate dashboard data.

Functional requirements include accepting comma-separated ingredients, avoiding duplicate selections, excluding optional ingredients from the coverage denominator, showing only matches of at least 75%, and supporting 100% ready-to-cook results. Input validation, parameterized SQL, transaction rollback and ownership checks are required across API mutations.

Layer	Technology	Reason for selection
Frontend	Next.js 16.4 React 19.3	One application for pantry input, results, details and account workflows.
Backend	Next.js Route Handlers on Node.js	Server-side validation, session checks, JSON APIs and database access.
Relational database	PostgreSQL 16	Normalized relationships, constraints, joins, routines, transactions and RLS.
Vector database	pgvector VECTOR(128)	Cosine similarity and vector indexing alongside the same recipe IDs.
Embedding model	Python scikit-learn TF-IDF and SVD	Local reproducible 128-dimensional vectors without an external inference API.
Database client and verification	node-postgres and Node test runner	Connection pooling, bound parameters and database/API regression checks.

pgvector satisfies the vector-database option in the project rubric. A separate MongoDB service is not used. The JSONB catalogue field preserves variable source metadata and instruction arrays, while relational keys continue to govern ingredient and user relationships. Semantic retrieval has a fixed vocabulary and may reject an unknown-vocabulary query.

ER Model

The business model has Users, Recipes and Ingredients as core entities. Ratings and AppSessions have their own identifiers. RecipeIngredients, UserPantry and SavedRecipes are associative, identification-dependent entities: their composite identifiers are formed from parent keys. There is no separate weak entity with a partial key in the conventional Chen sense.

Entity or association	Identifier	Main purpose
Users	user_id	User identity, role and dietary preference.
Recipes	recipe_id	Meal descriptions, instructions, status, vectors and source metadata.
Ingredients	ingredient_id	Canonical ingredient names and categories.
Ratings	rating_id	User verdict for a recipe; the user-recipe pair is unique.
AppSessions	token_hash	Opaque session digest, user link and expiry.
RecipeIngredients	recipe_id ingredient_id	Associates required or optional ingredients with quantities and units.
UserPantry	user_id ingredient_id	Associates a user with stored pantry ingredients.
SavedRecipes	user_id recipe_id	Associates users with bookmarked recipes.

Non simple and derived attributes

Ingredient lists are represented by association rows rather than a comma-separated attribute. The catalog_details JSONB attribute is structured: it contains arrays of ingredient lines and steps, source/license metadata, protein tags and an import source key. It is the principal non-simple stored attribute. The average rating, full-text search document, embedding and pantry match percentage are derived values. The generated search document is maintained by PostgreSQL; the embedding is regenerated by the backend when recipe content changes.

RecipeCatalogImports is administrative import-version metadata. It is included in the relational design but does not represent a kitchen-domain entity. Database nullable foreign keys are reflected in the schema diagrams; the API requires the user and recipe links for submitted ratings.

Entity Relationship Diagram

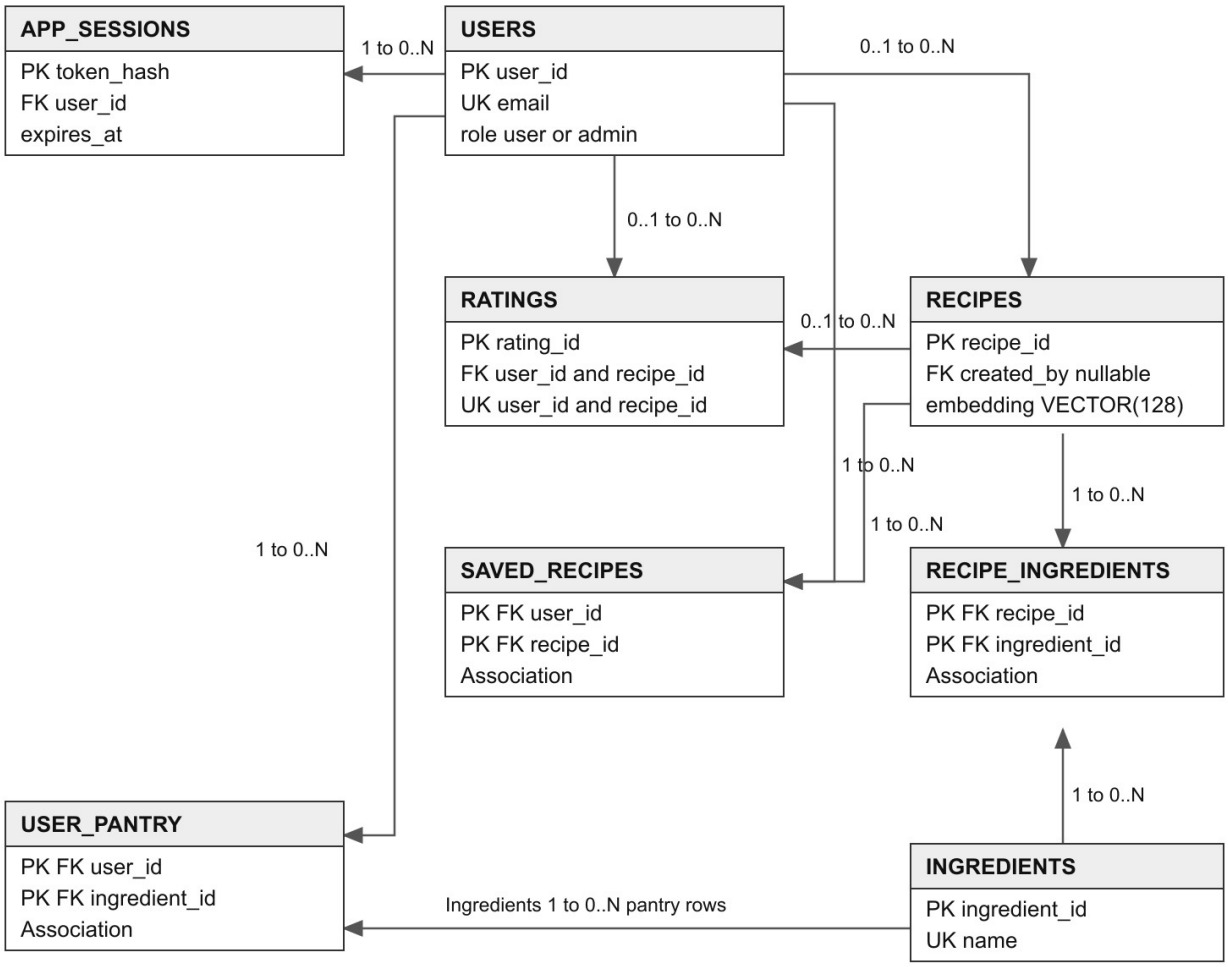


Figure 2 Parent-to-dependent arrows show stored cardinalities. Import metadata is independent.

A recipe may have no creator after user deletion because created_by uses SET NULL. Ingredient links, pantry entries, saves and sessions depend on their referenced parents. The API requires at least one required ingredient when creating a recipe; foreign keys alone do not enforce a minimum child count.

Relational Model

The schema contains nine base relations. Surrogate keys identify independent records; composite keys prevent duplicate relationship rows. Association relations are the tables introduced to represent many-to-many relationships rather than independent core entities.

Relation	Primary and foreign keys	Additional integrity
Users	PK user_id	Unique email and lower(email); required role limited to user or admin.
Recipes	PK recipe_id; created_by references Users	Nonempty title/method; valid status/difficulty; positive time when supplied.
Ingredients	PK ingredient_id	Required unique name.
RecipeIngredients	PK recipe_id ingredient_id; both are FKs	Required optional flag; positive quantity when supplied; parent deletion cascades.
UserPantry	PK user_id ingredient_id; both are FKs	Positive quantity when supplied; cascading FKs.
Ratings	PK rating_id; user_id and recipe_id are FKs	Unique user-recipe pair; required stars between 1 and 5.
SavedRecipes	PK user_id recipe_id; both are FKs	Duplicate saves prevented; cascading FKs.
AppSessions	PK token_hash; FK user_id	Required expiry; token digest must contain 64 hexadecimal characters.
RecipeCatalogImports	PK version	Required installed_at timestamp; supports idempotent imports.

RecipeIngredients, UserPantry and SavedRecipes are the association relations. RecipeCatalogImports is an implementation metadata relation outside the business ER model. Quantity columns are nullable in the installed schema; application recipe submissions require positive numeric quantities. Ratings foreign keys are nullable in SQL, while the API supplies and validates both links. These details are distinguished from stronger application-level rules.

Core Relational Schema

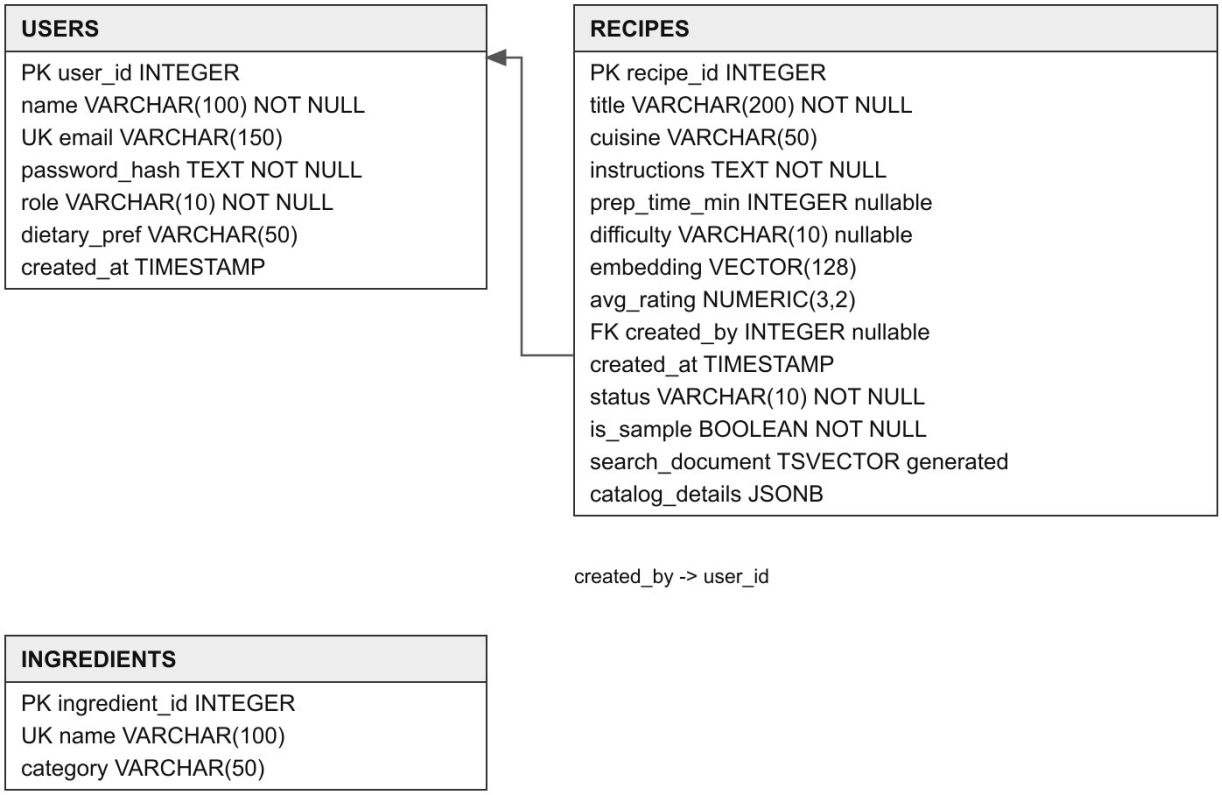


Figure 3 Core relations and the nullable creator reference. Attribute types match the installed database.

Dependent Relational Schema

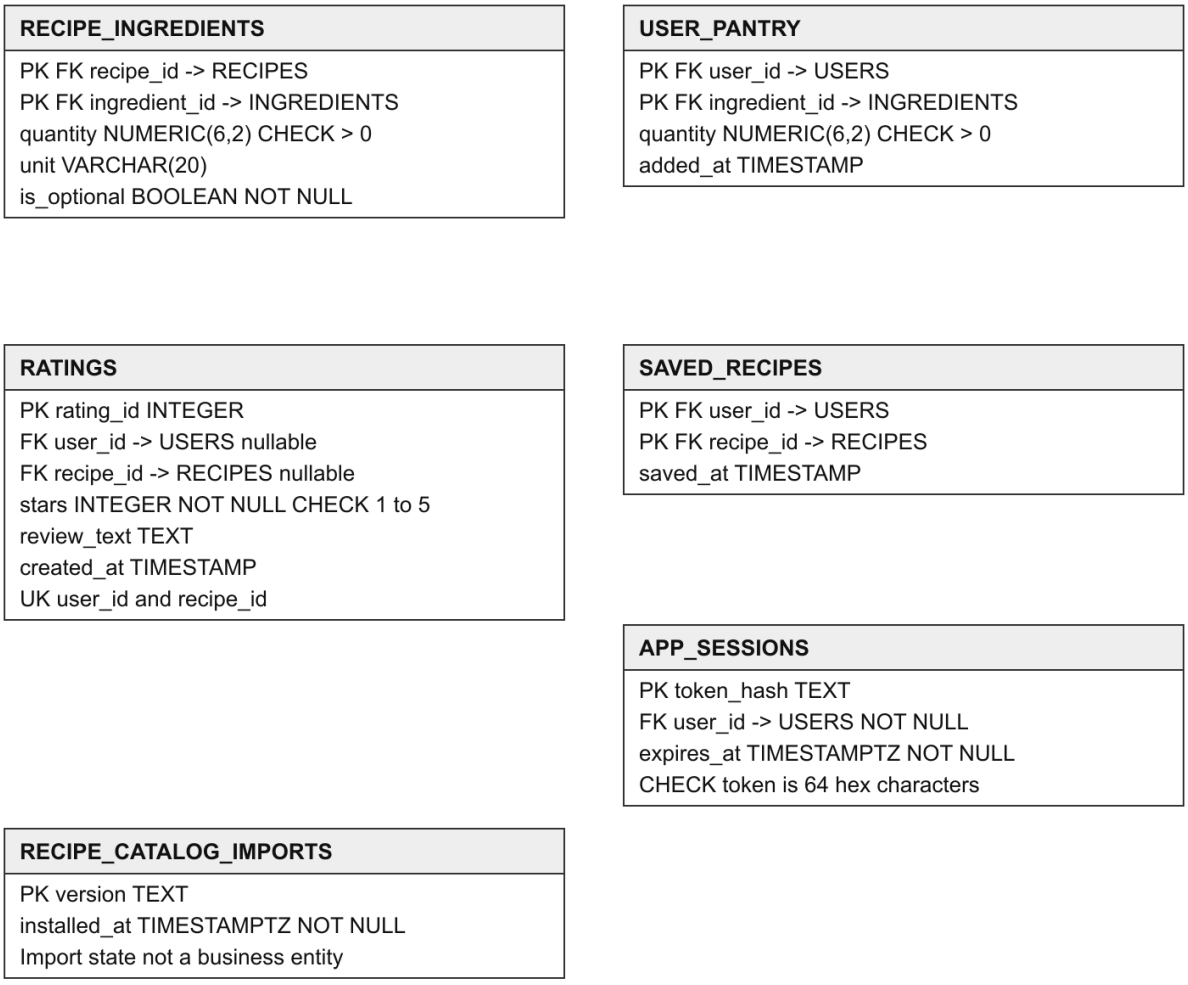


Figure 4 Association, rating, session and import metadata relations. FK targets are written beside each key.

SQL DDL Statements

The following core DDL illustrates the implemented relations and constraints. The complete setup is split across database/schema.sql, app.sql, integrity.sql, access.sql and the catalogue installer. Use the normal setup script for a fresh project database. The original schema.sql contains DROP TABLE statements and should not be rerun over populated user data.

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role VARCHAR(10) NOT NULL DEFAULT 'user'
    CHECK (role IN ('user','admin')),
  dietary_pref VARCHAR(50),
  created_at TIMESTAMP DEFAULT now()
);
CREATE TABLE ingredients (
  ingredient_id SERIAL PRIMARY KEY,
  name VARCHAR(100) UNIQUE NOT NULL,
  category VARCHAR(50)
);
CREATE TABLE recipes (
  recipe_id SERIAL PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  cuisine VARCHAR(50),
  instructions TEXT NOT NULL,
  prep_time_min INTEGER CHECK (prep_time_min > 0),
  difficulty VARCHAR(10)
    CHECK (difficulty IN ('Easy','Medium','Hard')),
  embedding VECTOR(128),
  avg_rating DECIMAL(3,2) DEFAULT 0,
  created_by INTEGER REFERENCES users(user_id)
    ON DELETE SET NULL,
  created_at TIMESTAMP DEFAULT now(),
  status VARCHAR(10) NOT NULL DEFAULT 'approved'
    CHECK (status IN ('approved','pending','rejected')),
  is_sample BOOLEAN NOT NULL DEFAULT true,
  catalog_details JSONB
);
```

The live schema also includes a generated weighted search_document column. Imported recipes can have unknown cooking times or difficulty, so these values remain nullable and are displayed honestly in the interface. Source metadata is stored in catalog_details rather than forcing every imported document into identical fields.

SQL DDL Relationships and Integrity

```
CREATE TABLE recipe_ingredients (
  recipe_id INTEGER REFERENCES recipes ON DELETE CASCADE,
  ingredient_id INTEGER REFERENCES ingredients ON DELETE CASCADE,
  quantity DECIMAL(6,2) CHECK (quantity > 0),
  unit VARCHAR(20), is_optional BOOLEAN NOT NULL DEFAULT false,
  PRIMARY KEY (recipe_id, ingredient_id)
);
CREATE TABLE user_pantry (
  user_id INTEGER REFERENCES users ON DELETE CASCADE,
  ingredient_id INTEGER REFERENCES ingredients ON DELETE CASCADE,
  quantity DECIMAL(6,2) CHECK (quantity > 0),
  added_at TIMESTAMP DEFAULT now(),
  PRIMARY KEY (user_id, ingredient_id)
);
CREATE TABLE ratings (
  rating_id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users ON DELETE CASCADE,
  recipe_id INTEGER REFERENCES recipes ON DELETE CASCADE,
  stars INTEGER NOT NULL CHECK (stars BETWEEN 1 AND 5),
  review_text TEXT, created_at TIMESTAMP DEFAULT now(),
  UNIQUE (user_id, recipe_id)
);
CREATE TABLE saved_recipes (
  user_id INTEGER REFERENCES users ON DELETE CASCADE,
  recipe_id INTEGER REFERENCES recipes ON DELETE CASCADE,
  saved_at TIMESTAMP DEFAULT now(),
  PRIMARY KEY (user_id, recipe_id)
);
CREATE TABLE app_sessions (
  token_hash TEXT PRIMARY KEY
    CHECK (token_hash ~ '^[a-f0-9]{64}$'),
  user_id INTEGER NOT NULL REFERENCES users ON DELETE CASCADE,
  expires_at TIMESTAMPTZ NOT NULL
);
CREATE TABLE recipe_catalog_imports (
  version TEXT PRIMARY KEY,
  installed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX idx_users_email_lower ON users(lower(email));
ALTER TABLE recipes ADD CONSTRAINT recipes_nonempty_text
  CHECK (length(trim(title)) >= 3
    AND length(trim(instructions)) >= 20);
```

These statements consolidate the final table shapes for explanation. Actual migrations add constraints idempotently. Composite keys prevent duplicate pantry, ingredient-link and bookmark rows. The foreign keys enforce valid references and cascading cleanup independently of frontend checks.

SQL DML Statements

The application performs INSERT, SELECT, UPDATE and DELETE through parameterized SQL. A regular user can submit a pending recipe, save a pantry and bookmark a meal; an admin can approve, edit or delete a recipe. The following demonstration creates a disposable recipe inside a transaction and rolls it back. It is a demonstration script, while the API workflow tests provide recorded execution evidence for the running application.

```
BEGIN;
CREATE TEMP TABLE report_demo_id(id INTEGER) ON COMMIT DROP;
WITH created AS (
  INSERT INTO recipes(title,instructions,created_by,status,is_sample)
  SELECT 'Report Demo Recipe',
    'Cook the selected ingredients and serve the meal warm.',
    user_id,'pending',false
  FROM users WHERE role='admin' ORDER BY user_id LIMIT 1
  RETURNING recipe_id
)
INSERT INTO report_demo_id SELECT recipe_id FROM created;
INSERT INTO recipe_ingredients
  (recipe_id,ingredient_id,quantity,unit,is_optional)
SELECT d.id,i.ingredient_id,1,'portion',false
FROM report_demo_id d CROSS JOIN ingredients i
WHERE i.name IN ('mushroom','garlic');
SELECT r.recipe_id,r.title,i.name,ri.quantity,ri.unit
FROM recipes r JOIN recipe_ingredients ri USING(recipe_id)
JOIN ingredients i USING(ingredient_id)
WHERE r.recipe_id IN (SELECT id FROM report_demo_id);
UPDATE recipes SET prep_time_min=15
WHERE recipe_id IN (SELECT id FROM report_demo_id);
DELETE FROM recipes
WHERE recipe_id IN (SELECT id FROM report_demo_id);
ROLLBACK;
```

Application transactions

The backend wraps recipe creation with all ingredient links in one transaction. A failed foreign key therefore rolls back the recipe as well. Editing replaces ingredient links and the embedding atomically. Pantry replacement uses a per-user transaction lock to avoid concurrent duplicate or lost writes. Ratings and saved recipes are linked to the authenticated user rather than an arbitrary user identifier supplied by the browser.

SQL Queries and Recursive Traversal

The pantry function uses conditional aggregates to count available required ingredients and collect missing IDs. The application applies the minimum match before counting results or paginating. Comparing integer counts avoids admitting a value below 75% just because it rounds upward.

```
WITH matched AS (
  SELECT * FROM pantry_match($1::INTEGER[])
)
SELECT r.recipe_id,r.title,m.required,m.matched,m.match_pct
FROM recipes r JOIN matched m USING(recipe_id)
WHERE r.status='approved' AND NOT r.is_sample
  AND (cardinality($1::INTEGER[])=0
       OR (m.required>0 AND m.matched*4>=m.required*3))
ORDER BY m.match_pct DESC NULLS LAST,r.recipe_id
LIMIT 24 OFFSET $2;
-- Correlated subquery and aggregation
SELECT r.title,
  (SELECT count(*) FROM ratings rt
   WHERE rt.recipe_id=r.recipe_id) AS rating_count
FROM recipes r WHERE r.status='approved';
SELECT cuisine,count(*) AS recipes
FROM recipes WHERE status='approved' AND NOT is_sample
GROUP BY cuisine HAVING count(*)>=5;
```

The following read-only recursive CTE traverses a stored instruction array in cooking order. It was executed against the current database and returned 13 rows for Jerk Lobster. This is an additional SQL demonstration; the normal recipe search API uses ordinary CTEs rather than this recursive traversal.

```
WITH RECURSIVE recipe_steps AS (
  SELECT recipe_id,title,catalog_details->'steps' AS steps,
         1 AS step_no
  FROM recipes WHERE title='Jerk Lobster'
    AND status='approved'
    AND jsonb_typeof(catalog_details->'steps')='array'
  UNION ALL
  SELECT recipe_id,title,steps,step_no+1
  FROM recipe_steps
  WHERE step_no<jsonb_array_length(steps)
)
SELECT step_no,steps->>(step_no-1) AS instruction
FROM recipe_steps ORDER BY step_no;
```

SQL Functions Procedures and Triggers

pantry_match is the stored function used by the web application. It returns required and matched counts, missing ingredient IDs and a percentage. Optional ingredient links are excluded from the denominator. cuisine_summary is a grouped view used for administrative summaries.

```
SELECT * FROM pantry_match(ARRAY[1,16,17,18,36])
ORDER BY match_pct DESC NULLS LAST LIMIT 24;
CREATE OR REPLACE VIEW cuisine_summary AS
SELECT cuisine,count(*) AS recipe_count,
       round(avg(avg_rating),2) AS average_rating
FROM recipes WHERE status='approved' GROUP BY cuisine;
CREATE OR REPLACE FUNCTION fn_update_avg_rating()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP <> 'DELETE' THEN
    UPDATE recipes SET avg_rating=coalesce((
      SELECT round(avg(stars)::numeric,2) FROM ratings
      WHERE recipe_id=NEW.recipe_id),0)
    WHERE recipe_id=NEW.recipe_id;
  END IF;
  IF TG_OP='DELETE' OR (TG_OP='UPDATE' AND
      OLD.recipe_id IS DISTINCT FROM NEW.recipe_id) THEN
    UPDATE recipes SET avg_rating=coalesce((
      SELECT round(avg(stars)::numeric,2) FROM ratings
      WHERE recipe_id=OLD.recipe_id),0)
    WHERE recipe_id=OLD.recipe_id;
  END IF;
  RETURN NULL;
END; $$;
CREATE TRIGGER trg_avg_rating
AFTER INSERT OR UPDATE OR DELETE ON ratings
FOR EACH ROW EXECUTE FUNCTION fn_update_avg_rating();
```

access.sql gives the rating-maintenance function narrowly scoped owner rights, so rating updates can maintain the recipe average while ordinary users cannot edit recipe records. The function handles both the old and new recipe when a rating is moved.

The inherited add_recipe_with_ingredients procedure inserts a recipe and loops through ingredient arrays. It includes an internal COMMIT, so a demonstration CALL must run at top level in a disposable database, not inside a BEGIN/ROLLBACK block. The web API instead owns the transaction boundary and does not depend on this legacy procedure.

```
CALL add_recipe_with_ingredients(
  'Demo Recipe','American','Prepare and cook the demo meal.',
  20,'Easy',1,ARRAY[1,2,3]::INTEGER[],
  ARRAY[1,1,1]::NUMERIC[],ARRAY['cup','cup','cup']::VARCHAR[]
);
```

NoSQL Vector Database Commands

PantryChef uses pgvector as the vector-database option. The recipe table stores normalized VECTOR(128) embeddings. Training concatenates title, instructions and ingredient names, fits a TF-IDF vocabulary and reduces it with TruncatedSVD. Node.js uses the same exported model for user queries and recipe edits. A fixed local vocabulary limits the range of phrases the model can understand.

```
CREATE EXTENSION IF NOT EXISTS vector;
-- The column is included in the core recipe DDL.
CREATE INDEX idx_recipes_embedding ON recipes
USING ivfflat (embedding vector_cosine_ops) WITH (lists=50);
CREATE INDEX idx_recipe_ing_ingredient
  ON recipe_ingredients(ingredient_id);
-- Read nearest recipes using an existing stored vector.
WITH anchor AS (
  SELECT embedding FROM recipes
  WHERE status='approved' AND NOT is_sample
    AND embedding IS NOT NULL
  ORDER BY recipe_id LIMIT 1
)
SELECT r.recipe_id,r.title,
       round((1-(r.embedding <=> a.embedding))::numeric,3)
         AS similarity
FROM recipes r CROSS JOIN anchor a
WHERE r.status='approved' AND NOT r.is_sample
ORDER BY r.embedding <=> a.embedding LIMIT 24;
-- Backend-bound update after embedding the edited content.
UPDATE recipes SET embedding=$1::vector
WHERE recipe_id=$2;
-- Admin deletion removes the vector-bearing recipe row.
DELETE FROM recipes WHERE recipe_id=$1;
```

Meaning search reads recipes by cosine distance. Editing recomputes the embedding from the edited text and ingredient names. Deletion removes the vector-bearing recipe and cascades to dependent rows. Integration tests verify the changed vector against the expected embedding and check its absence after deletion. The backend binds validated values to the UPDATE and DELETE parameters.

Full text search

```
CREATE INDEX idx_recipes_search
  ON recipes USING gin(search_document);
SELECT recipe_id,title FROM recipes
WHERE search_document @@
  websearch_to_tsquery('english','chicken rice');
```

The generated search document weights titles above instructions. The historical benchmark demonstrates index use; filtered queries and small tables can use other plans.

Authentication and Database Authorization

Regular User and Admin are application roles backed by database policies. Passwords are salted and hashed using scrypt. Opaque session tokens are stored as SHA-256 digests with an expiry; the browser receives an HttpOnly, SameSite cookie. Registration always creates a regular user even if a client attempts to request the admin role.

Ordinary requests use pantrychef_runtime, a non-owner PostgreSQL login without superuser or BYPASSRLS privileges. Each transaction sets only the server-derived session digest. Database helper functions resolve the user and role from a valid session. Request-supplied user IDs or role strings are not trusted for ownership or administrative rights.

```
ALTER TABLE recipes ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_pantry ENABLE ROW LEVEL SECURITY;
ALTER TABLE saved_recipes ENABLE ROW LEVEL SECURITY;
ALTER TABLE ratings ENABLE ROW LEVEL SECURITY;
CREATE POLICY pantry_owner ON user_pantry
FOR ALL TO pantrychef_runtime
USING (user_id=app_actor_id())
WITH CHECK (user_id=app_actor_id());
CREATE POLICY recipe_delete ON recipes
FOR DELETE TO pantrychef_runtime USING (app_is_admin());
REVOKE ALL ON ALL TABLES IN SCHEMA public
FROM pantrychef_runtime;
GRANT SELECT,INSERT,UPDATE,DELETE ON
  recipes,recipe_ingredients,user_pantry,ratings,saved_recipes
TO pantrychef_runtime;
```

The complete policies in database/access.sql also govern recipe visibility, pending submissions, recipe links, rating ownership and public profiles. The owner connection is restricted to authentication and maintenance operations and remains a server-side trust boundary. Direct database tests reject role promotion, password/session reads, schema changes, owner-role assumption and cross-user access.

Configuration and local execution

```
# Open the extracted PantryChef-App folder in VS Code.
powershell -ExecutionPolicy Bypass -File .\start-app.ps1
# The app opens at http://127.0.0.1:3000/
# Fresh setup uses Node.js 22+, Python 3.11+, PostgreSQL + pgvector.
npm.cmd install
python -m pip install -r requirements.txt
npm.cmd run db:setup
npm.cmd run dev
```

The supplied Windows launcher manages the local WSL PostgreSQL lifecycle. Database URLs, generated model data and credentials are kept outside the public repository. The report contains no connection strings, passwords or session tokens.

Results

Read only database check	Observed result
Total stored recipe rows	1449
Approved public non-synthetic recipes	545
Public recipes with a stored embedding	545
Attributed source imports in public catalogue	500
Base relations	9
Recursive Jerk Lobster instruction rows	13
Latest automated tests	18 passed 0 failed

Database counts were read from the current project database on 2 November 2026. Stored rows include synthetic benchmark and nonpublic records, so the full table count is different from the public discovery count. The next table shows actual SELECT output from the public catalogue, rather than invented sample records.

Recipe ID	Title	Cuisine
100597	Pasties	International
100596	Pasta, Meat and Cheese Casserole (Mostaccioli)	International
100595	Nigerian Rice with Stew	Nigerian
100582	Phá»Ÿ GÃ (Vietnamese Chicken Noodle Soup)	International
100581	Peri-Peri Chicken	International

Recursive query output

Step	Instruction excerpt
1	Thinly slice the scallions.
2	Melt the butter in a small skillet.

Validation and Performance Results

The latest recorded production run passed all 18 automated tests. The test suite exercises API workflows against PostgreSQL, including direct runtime-role denials, transactional failures, catalogue provenance, vector regeneration/deletion and image coverage. Browser checks separately verified the 75% result, missing ingredient text, photo consistency and the recipe detail view.

Test group	Verified behavior
Database access	Ownership and visibility policies; forbidden user-role, schema and credential access.
Integration and transactions	Registration/login, saves, pantry, ratings, moderation, recipe editing/deletion and atomic rollback.
Catalogue and search	Five hundred distinct imported methods, all seven protein categories, real required ingredient guards and dietary exclusions.
Photo coverage	All 545 public recipes use different local image paths, source URLs and file hashes.
Pantry boundary	75% and 100% included; 50% and 70% excluded; multi-page totals and ready-only behavior checked.
Security validation	Salted passwords, malformed input handling and origin checks.

```
npm.cmd test
# Recorded summary
# tests 18
# pass 18
# fail 0
npm.cmd run build
# Production build completed successfully.
```

Historical EXPLAIN ANALYZE evidence in docs/QUERY-PLANS-100K.md covers a separate 100,000-row synthetic load test. In that recording, full-text lookup used the GIN index and completed in approximately 1.015 ms; vector nearest-neighbor search used IVFFlat and completed in approximately 1.448 ms. Those timings precede later security and search changes and are observations on that test environment, not latency guarantees for the current API.

Current recipe searches disable LLVM JIT only inside the search transaction because recorded compilation time outweighed query execution. The 75% condition is applied within SQL before the window count, ordering and LIMIT/OFFSET; result totals therefore reflect qualifying recipes.

Next.js Project Tree Structure

```
PantryChef-App/
  app/
    api/[...path]/route.js     JSON API and role-checked mutations
    page.js                   Pantry and recipe interface
    layout.js                 Root application layout
    globals.css               Responsive warm-color theme
    motion-ui.js              Accessible interaction transitions
  lib/
    db.mjs                    Pools transactions session context
    auth.mjs                  Current user and admin guards
    security.mjs              Hashing tokens and validation
    recipes.mjs               Search and detail queries
    ingredient-intent.mjs     Required/excluded ingredient guards
    proteins.mjs              Seven protein ingredient groups
    embeddings.mjs            Local fitted query embedding
    cooking-guide.mjs         Recipe method presentation
    recipe-photos.mjs         Stable image lookup
  data/
    meat-recipes.mjs           500 attributed source recipes
    indian-recipes.mjs         40 established Indian dishes
    food-photos.mjs            Photo source and license records
    source-recipe-photos.mjs   Stable imported recipe assignments
  database/
    schema.sql                Base DDL and legacy routines
    app.sql                   Search function trigger sessions view
    integrity.sql             Additional integrity constraints
    access.sql                Runtime grants and RLS policies
    demo_queries.sql          SQL demonstration queries
  scripts/
    setup.mjs                 Idempotent database setup
    train_embeddings.py       TF-IDF and SVD training
    meat-catalog.mjs           Source import installation
    scale.mjs                 Separate synthetic scale test
    serve.mjs                 Server and database lifecycle
  tests/                      Six test files eighteen test cases
  docs/                       Design tests plans screenshots
  public/food/                Locally bundled image files
  public/photo-credits.html   Image attribution catalogue
  .env.example                Placeholder configuration only
  package.json                Dependencies and run commands
  start-app.ps1               Windows launcher
```

The frontend and backend are part of one Next.js application. A separate Python web server is not required; Python is used for model training during setup. Private .env.local and .generated files are excluded from version control and the sharing archive.

Application Code and Demo

These source figures are rendered from the current project files at revision 7ee532e. The search predicate enforces the exact pantry threshold in the database. The authentication guard rejects anonymous requests and prevents regular users from invoking admin-only actions.

lib/recipes.mjs

```
23     SELECT 1 FROM recipe_ingredients pi JOIN ingredients ip USING(ingredient_id)
     WHERE pi.recipe_id=r.recipe_id AND ip.name IN(SELECT
     jsonb_array_elements_text(g)))) AND ($3='' OR r.cuisine=$3)
24    AND (NOT $5::boolean OR (m.required=m.matched AND m.required>0))
25    AND (cardinality($1::integer[])=0 OR (m.required>0 AND
     m.matched*4>=m.required*3))
26    AND ($7::integer IS NULL OR EXISTS(SELECT 1 FROM saved_recipes s WHERE
     s.recipe_id=r.recipe_id AND s.user_id=$7))
27    AND ($4='' OR NOT EXISTS(SELECT 1 FROM recipe_ingredients dx JOIN ingredients
     ix USING(ingredient_id)
28    WHERE dx.recipe_id=r.recipe_id AND NOT dx.is_optional AND (
```

Figure 5 SQL predicates include the 75% required-ingredient threshold before pagination.

lib/auth.mjs

```
8    }
9    export async function requireUser(admin=false) {
10    const user=await currentUser();if(!user)throw new ApiError('Please sign in
     first.',401);
11    if(admin&&user.role!=='admin')throw new ApiError('Administrator access
     required.',403);return user;
12   }
```

Figure 6 Server-side current-user and administrator authorization guard.

The API validates request shapes and origins before issuing parameterized SQL. Database policies then enforce access independently of interface visibility. Recipe edits update the embedding and ingredient rows within one transaction.

Application Demo Recipe Discovery

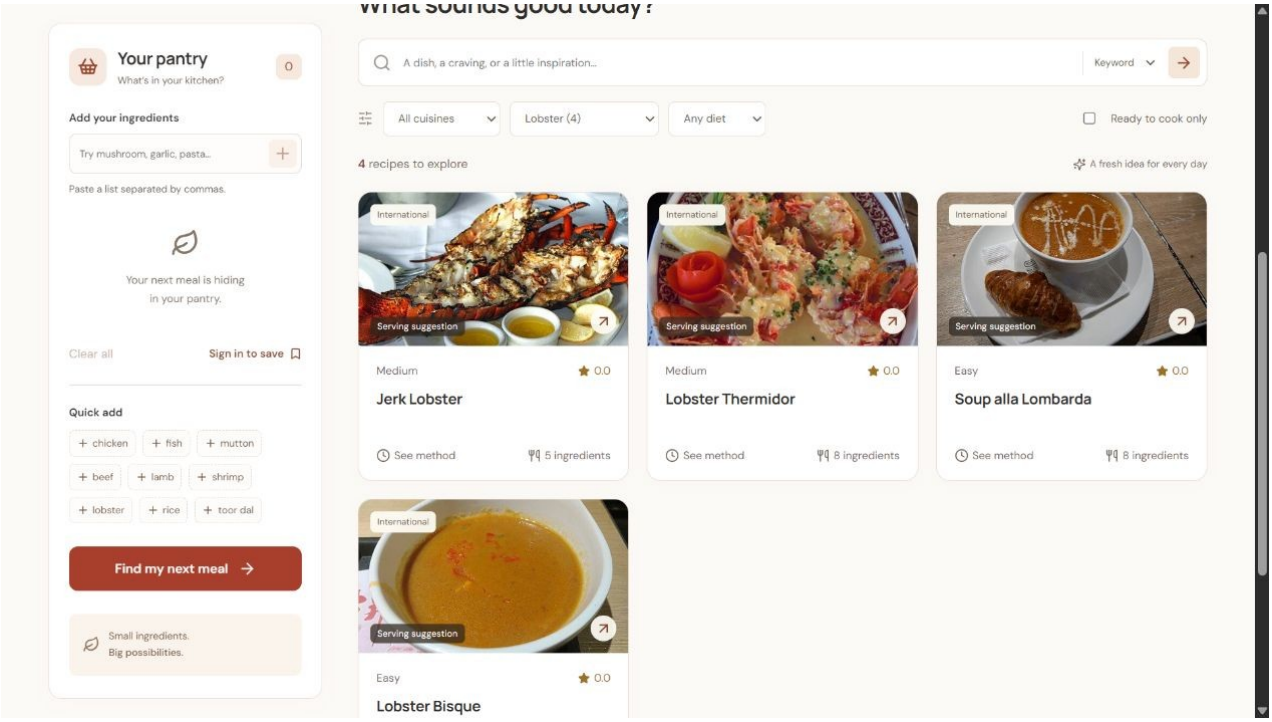


Figure 7 The expanded lobster collection with four distinct locally hosted serving-suggestion images.

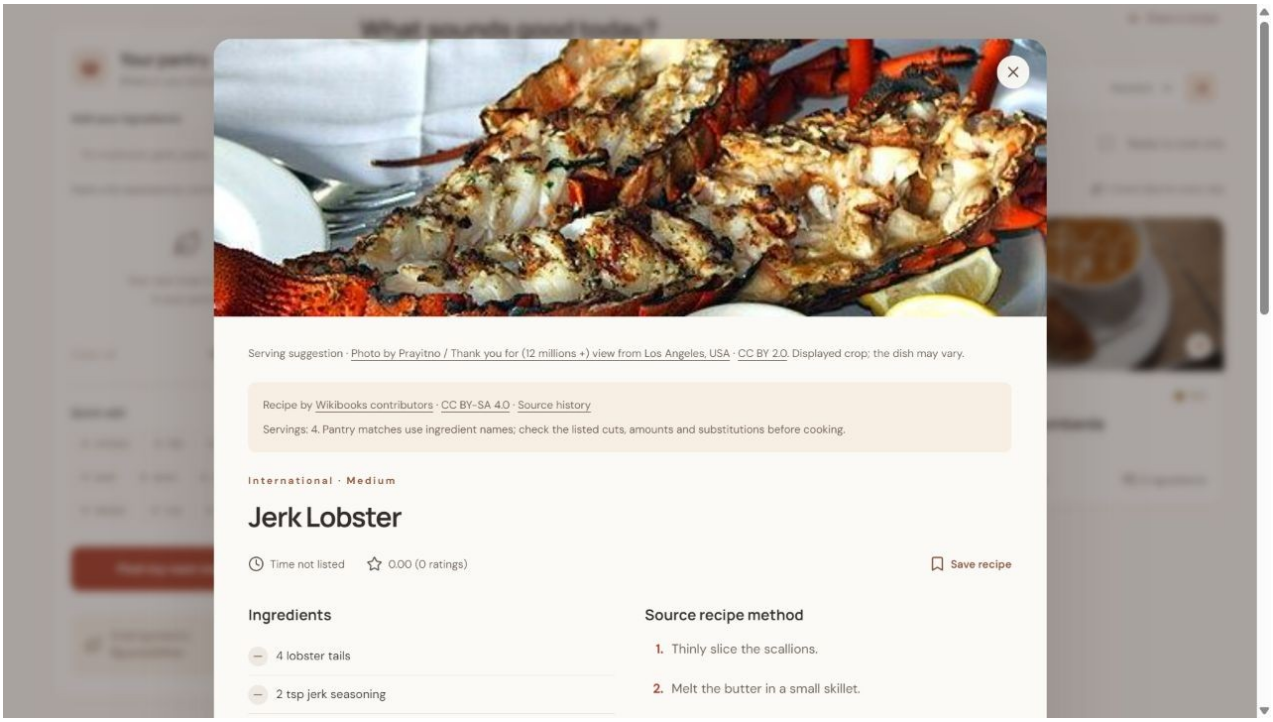


Figure 8 Recipe detail with image attribution, original source/license, ingredients and cooking method.

Application Demo Pantry Match

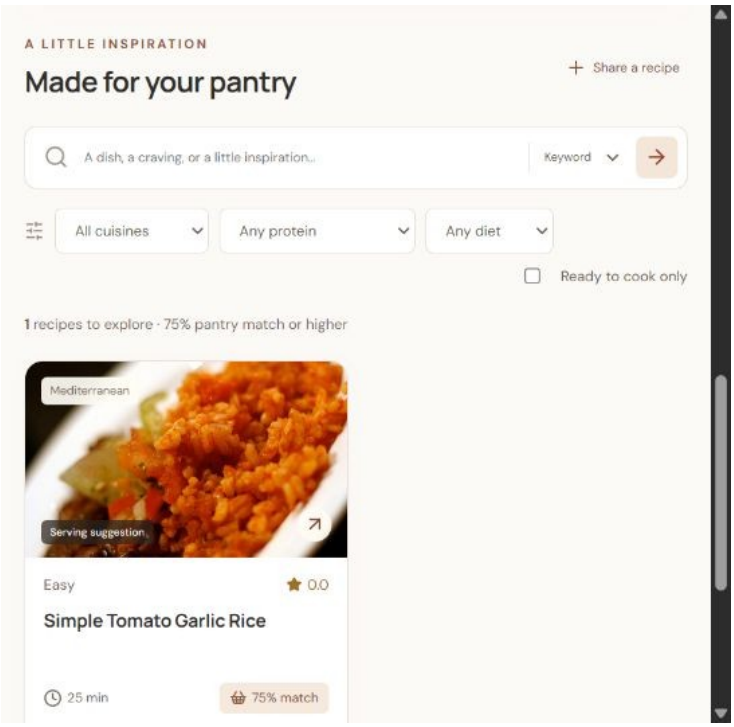


Figure 9 Exact 75% match retained while lower matches are excluded; onion is listed as missing.

Selecting ingredients and clicking Find my next meal applies a minimum coverage of 75% of the recipe's required ingredient names. Ready to cook only requires 100%. Selecting additional cuisine, protein or dietary filters narrows the same qualifying result set. Optional ingredients do not reduce the percentage.

Food photos illustrate serving suggestions and may differ from the recipe ingredients or presentation. Recipe instructions imported from Wikibooks retain CC BY-SA 4.0 attribution. Image licenses are separate and are recorded with each image in the public credits catalogue.

Evaluation Rubrics Completion Status

This status records implementation and validation evidence against the supplied 50-mark rubric. Marks are allocated by the evaluator. Individual originality, contribution and modification/viva performance require student demonstration and are not certified by software tests.

Criterion	Max	Completion evidence	Status
Database Design	5	Entity diagram, relations, keys, FKs and integrity rules.	Included
Database Creation	9	PostgreSQL DDL, pgvector and technology justification.	Implemented
SQL Operations and Queries	8	DML, joins, correlated queries and an executed recursive step traversal.	Demonstrated
Advanced Database Concepts	13	Procedure/function, trigger, CTE, vector search, vector update and deletion.	Implemented and tested
Web Application and Database Integration	5	Next.js interface, Route Handlers and parameterized DB integration.	Implemented and tested
Authentication and Authorization	4	User/Admin roles, hashed sessions, database grants and RLS tests.	Implemented and tested
Documentation GitHub Testing and Originality	6	Report/source/tests present. Originality, real individual work and viva remain faculty-assessed.	Documentation ready; individual assessment pending

Student contribution and assistance disclosure

Team members are Tarun Jagan and Tanishk Gowda. The implementation and report were prepared with Codex assistance. Each student is responsible for understanding the design, demonstrating genuine contributions and explaining or modifying the system during evaluation. The report does not assign equal commits or certify an individual contribution that is not evidenced in the repository.

Limits and future improvements

Pantry coverage checks names, not available quantities or allergen safety. Imported source instructions are preserved and are not independently kitchen-tested. Semantic matching is limited by the fitted vocabulary. Future work includes stronger quantity/unit handling, broader licensed recipe sources, improved semantic models and deployment hardening. An identified Next.js dependency advisory remains a maintenance task; this report does not claim it has been patched.

Evidence and References

Project source repository https://github.com/DarkTJ7/DBMS-MINI-PROJECT. The implementation snapshot used for this report is commit 7ee532e. Complete SQL definitions remain in the tracked database files; the report excerpts explain the design rather than replace the setup workflow.

database/schema.sql app.sql integrity.sql access.sql and demo_queries.sql - DDL, routines, integrity and permissions.

lib/recipes.mjs lib/embeddings.mjs app/api/[...path]/route.js and lib/auth.mjs - search, vectors, API integration and guards.

tests/*.test.mjs and docs/PANTRY-MATCH-TEST-OUTPUT.txt - eighteen passing recorded tests.

docs/TESTING.md and docs/QUERY-PLANS-100K.md - verification history and qualified scale measurements.

data/meat-recipes.mjs and docs/RECIPE-SOURCES.md - Wikibooks source URLs, original ingredient lines, methods and recipe licenses.

public/photo-credits.html and docs/PHOTO-CREDITS.md - individual image authors, original Commons sources and licenses.

Supplied PES University DBMS Level 3 rubric - evaluation criteria totaling fifty marks.

Photo Credits for Displayed Captures

Grilled Lobster (8558909573) - Prayitno / Thank you for (12 millions +) view from Los Angeles, USA. CC BY 2.0. https://commons.wikimedia.org/wiki/File:Grilled_Lobster_(8558909573).jpg License https://creativecommons.org/licenses/by/2.0. Displayed as a resized/cropped application screenshot.

2014-06-12 Lobster Thermidor, Pepperpot, West Runton - West Rünton / Kofolom (Kofolom). CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:2014-06-12_Lobster_Thermidor,_Pepperpot,_West_Runton.jpg License https://creativecommons.org/licenses/by-sa/4.0. Displayed as a resized/cropped application screenshot.

A Lobster Soup and Croissant - Peachyeung316. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:A_Lobster_Soup_and_Croissant.jpg License https://creativecommons.org/licenses/by-sa/4.0. Displayed as a resized/cropped application screenshot.

Lobster Bisque in Hong Kong Fast Food Shop - Ceeseven. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Lobster_Bisque_in_Hong_Kong_Fast_Food_Shop.jpg License https://creativecommons.org/licenses/by-sa/4.0. Displayed as a resized/cropped application screenshot.

Mexican tomato rice and beans - Maryam S.. CC BY 2.0. https://commons.wikimedia.org/wiki/File:Mexican_rice_and_beans.jpg License https://creativecommons.org/licenses/by/2.0. Displayed as a resized/cropped application screenshot.